

“Recommender System with Frank-Wolfe Method” final project report

Shabnam Sattar[†]

Abstract—Matrix completion is pivotal in recommender systems and various other domains where data is only partially observed. In this work, we implement and evaluate the Frank-Wolfe algorithm for matrix completion under nuclear-norm constraints. Our approach iteratively constructs low-rank approximations by utilizing a linear minimization oracle (LMO), which efficiently finds the top singular vectors to guide each update. We compare two step-size strategies—diminishing step-size and exact line search—on two slimmed benchmark datasets, MovieLens and X-Wines. While both methods show convergence, the line search strategy achieves higher accuracy on X-Wines but incurs a higher computational cost. Furthermore, the results highlight sensitivity to class imbalance in the datasets, suggesting that richer sampling strategies or advanced variants of Frank-Wolfe may further enhance accuracy.

I. INTRODUCTION

Matrix completion aims to recover missing entries in a partially observed matrix, often under the assumption of low rank. This problem has significant relevance in recommender systems, where user-item rating matrices typically remain sparse. By exploiting a low-dimensional structure, one can infer missing ratings and improve the quality of recommendations. Beyond recommender systems, matrix completion applies to machine learning, computer vision, and collaborative filtering, particularly in tasks that demand efficient recovery from limited observations [2].

A classic approach formulates the problem by minimizing the sum of squared errors over only the observed entries. However, enforcing a strict rank constraint is computationally challenging because the rank function is non-convex. Instead, a convex surrogate in the form of the nuclear norm—i.e., the sum of the singular values—is frequently employed. This transforms the original formulation into a convex optimization problem, making it amenable to established solvers and theoretical guarantees.

Among the broad family of optimization strategies, the Frank-Wolfe (FW) algorithm stands out for handling constrained problems without requiring orthogonal projections. Originally introduced by Frank and Wolfe in 1956 [3], FW iteratively seeks a descent direction by solving a linear subproblem with respect to the gradient. When coupled with nuclear-norm constraints, the linear subproblem reduces to finding the top singular vector pair of the negative gradient, providing a rank-one update direction at each step. This rank-one structure is especially compelling for large matrices, as it

can exploit efficient numerical routines to estimate dominant singular vectors.

In this project, we implement the Frank-Wolfe algorithm for matrix completion tasks and compare two different step-size rules: (i) diminishing step-size, which follows a preset schedule decreasing at each iteration; and (ii) exact line search, which adaptively determines the optimal step-size via direct minimization of the loss. We evaluate both approaches on two curated datasets—MovieLens Slim and X-Wines Slim—designed for reduced sparsity and practical demonstration. By measuring metrics such as loss convergence, R^2 , duality gap, and CPU time, we highlight strengths and limitations of each method.

The remainder of this report is structured as follows. We first review the fundamentals of matrix completion and the nuclear-norm-constrained formulation. We then present the Frank-Wolfe algorithm, detailing its step-size selection strategies and computation of the duality gap. Next, we outline our implementation and experimental setup, followed by illustrative results on the two datasets. We conclude by discussing the performance trade-offs, the implications of dataset imbalance, and potential areas for enhancing matrix completion frameworks in recommender systems.

II. MATRIX COMPLETION

Matrix completion is a key technique in recommender systems and many other areas of science and engineering. The central idea is to reconstruct a *low-rank* matrix $X \in \mathbb{R}^{n_1 \times n_2}$ from observations of only a sparse subset of its entries. In a recommender-system setting, for instance, X_{ij} could represent user i 's rating of item j . Because many users rate only a small fraction of the available items, one seeks to estimate the missing entries of X by imposing a low-rank constraint, thereby capturing the underlying structure in the user preferences. [1]

Objective Function: A common way to formalize this problem is to minimize the squared error over the observed entries [4]:

$$\min_{X \in \mathbb{R}^{n_1 \times n_2}} \sum_{(i,j) \in J} (X_{ij} - U_{ij})^2 \quad \text{st.} \quad \text{rank}(X) \leq \delta$$

where $\{U_{ij}\}_{(i,j) \in J}$ are the known entries and $\delta > 0$ reflects the assumed rank of the completed matrix. In practice, instead of directly enforcing $\text{rank}(X) \leq \delta$, one often replaces the rank constraint by a nuclear-norm constraint, $\|X\|_* \leq \delta$, because

[†]Department of Mathematics, University of Padova, email: {shabnam.sattar}@studenti.unipd.it

the nuclear norm (i.e., the sum of the singular values of X) is a *convex* surrogate for the non-convex rank function. [1]

With this nuclear-norm formulation,

$$\min_{X \in \mathbb{R}^{n_1 \times n_2}} \sum_{(i,j) \in J} (X_{ij} - U_{ij})^2 \quad \text{st.} \quad \|X\|_* \leq \delta$$

the feasible set becomes the convex hull of rank-one matrices, namely:

$$C = \{X : \|X\|_* \leq \delta\} = \text{conv}\{\delta u v^T : \|u\| = \|v\| = 1\}$$

An iterative Frank-Wolfe algorithm can then be applied, where each step requires solving a Linear Minimization Oracle:

$$\text{LMO}_C(\nabla f(X_k)) \in \arg \min_{\|X\|_* \leq \delta} \left\{ \text{tr}(\nabla f(X_k)^T X) \right\}$$

Gradient: The gradient of the objective function for the observed entries is given by:

$$\nabla f(X) = 2(X - U)_J,$$

where $(X - U)_J$ represents the matrix coinciding with $X - U$ on the observed indices J , with all other entries set to zero. This gradient is then used in conjunction with a linear minimization oracle (LMO) in the Frank-Wolfe algorithm to iteratively improve the solution.

LMO: The solution to the LMO boils down to finding the top singular vector pair of $-\nabla f(X_k)$. If u_1 and v_1 denote the left and right singular vectors corresponding to the largest singular value of $-\nabla f(X_k)$, then the optimal rank-one update is given by $\delta u_1 v_1^T$. Iterating this procedure leads the FW method to build an approximate solution as a sparse sum of rank-one matrices. In the context of recommender systems, this approach efficiently uncovers latent factors (encoded by u_1 and v_1 at each step) that best explain the observed user-item ratings, thus yielding an effective low-rank completion of the rating matrix. [1]

III. CLASSICAL FRANK-WOLFE ALGORITHM

Frank-Wolfe: The Frank-Wolfe algorithm, introduced by Marguerite Frank and Philip Wolfe in 1956 [3], is a classical optimization method to minimize a smooth and convex objective function over a compact convex feasible set. It is particularly notable for its ability to avoid projections onto the feasible set, relying instead on a linear minimization oracle (LMO) to iteratively compute feasible descent directions.

The general optimization problem can be stated as:

$$\min_{x \in C} f(x),$$

where $f(x)$ is a convex and continuously differentiable function and C is a compact convex set. At each iteration k , the algorithm uses the LMO to solve the linear subproblem:

$$s_k = \text{LMO}_C(\nabla f(x_k)) \in \arg \min_{z \in C} \langle \nabla f(x_k), z \rangle.$$

The descent direction is then defined as:

$$d_k^{FW} = s_k - x_k,$$

and the next iteration is calculated as:

$$x_{k+1} = x_k + \alpha_k d_k^{FW},$$

where $\alpha_k \in [0, 1]$ is the step size, chosen by a line search or a predefined formula.

Exact Line Search in Frank-Wolfe: Here we provide an explanation of how the step size is found in the Frank-Wolfe line search step for matrix completion. [5] [6]

At iteration t , we have:

- A current iterate x_t .
- A direction $d_t = s_t - x_t$, where s_t is given by the Linear Minimization Oracle (LMO).
- A set of training entries $(i, j) \in \text{Train}$, with corresponding true ratings U_{ij} .
- A loss function on these entries of the form

$$f(X) = \sum_{(i,j) \in \text{Train}} (X_{ij} - U_{ij})^2.$$

We seek a step size $\alpha \in [0, 1]$ that solves

$$\min_{\alpha \in [0, 1]} f(x_t + \alpha d_t).$$

Now we define A and B like here:

$$A_{ij} = x_t[i, j] - U_{ij}$$

$$B_{ij} = d_t[i, j] = s_t[i, j] - x_t[i, j]$$

Then each squared error term for $(i, j) \in \text{Train}$ becomes:

$$(x_t[i, j] + \alpha B_{ij} - U_{ij})^2 = (A_{ij} + \alpha B_{ij})^2.$$

Therefore, the total training loss is:

$$F(x_t + \alpha d_t) = \sum_{(i,j) \in \text{Train}} (A_{ij} + \alpha B_{ij})^2.$$

To find the α that minimizes the above sum, we set its derivative with respect to α to zero:

$$\frac{d}{d\alpha} \sum_{(i,j) \in \text{Train}} (A_{ij} + \alpha B_{ij})^2 =$$

$$2 \sum_{(i,j) \in \text{Train}} (A_{ij} + \alpha B_{ij}) B_{ij} = 0.$$

Rearranging, we obtain

$$\sum_{(i,j)} B_{ij} A_{ij} + \alpha \sum_{(i,j)} B_{ij}^2 = 0 \implies \alpha = - \frac{\sum_{(i,j)} A_{ij} B_{ij}}{\sum_{(i,j)} B_{ij}^2}.$$

Since Frank-Wolfe restricts $\alpha \in [0, 1]$ (to maintain feasibility in convex settings), we clip this value to the interval $[0, 1]$. If the denominator $\sum B_{ij}^2$ is zero, then we simply set $\alpha = 0$.

Dual Gap: The duality gap provides a certificate for the quality of a solution in constrained convex optimization. For a feasible point $x \in C$, it is defined as:

$$g(x) = \max_{s \in C} \langle x - s, \nabla f(x) \rangle.$$

This gap represents the difference between the linearized objective function $f(x) + \langle \nabla f(x), s - x \rangle$ and the true objective over the feasible set C . By convexity, $g(x) \geq f(x) - f(x^*)$, where $f(x^*)$ is the optimal value.

In the Frank-Wolfe algorithm, $g(x)$ is computed at each iteration as a by-product of solving the linear sub-problem. This makes it a useful stopping criterion and a practical tool for certifying convergence.

IV. IMPLEMENTATION

We implement the classic Frank-Wolfe algorithm, as shown in Algorithm 1. In the following, we outline the key details of implementation.

A. Starting Point $x^{(0)}$

The initialization of the algorithm begins with a randomly generated matrix whose nuclear norm does not exceed δ . To achieve this, we construct two *extreme* matrices by taking the outer product of randomly drawn, normalized vectors for both rows and columns. Each of these matrices independently attains a nuclear norm of δ . A convex combination of these two matrices is then taken, where a random weight determines the contribution of each matrix. This ensures that the resulting matrix maintains a nuclear norm within the feasible set, making it a valid starting point for the optimization process.

B. Linear Minimization Oracle (LMO)

The Linear Minimization Oracle (LMO) in the Frank-Wolfe algorithm requires solving a subproblem that reduces to finding the leading singular vector pair of $-\nabla f(X_k)$. If u_1 and v_1 represent the left and right singular vectors corresponding to the largest singular value of $-\nabla f(X_k)$, then the optimal rank-one update is given by:

$$\delta u_1 v_1^T$$

C. Step Size γ

For the step size selection in the Frank-Wolfe algorithm, we employ two different strategies:

- 1) **Diminishing Step Size:** A predefined step size of $\frac{2}{i+1}$ is used, where i represents the iteration index. This ensures a gradual reduction in step size over iterations, facilitating convergence.
- 2) **Exact Line Search:** In this approach, the step size is chosen dynamically at each iteration by minimizing the objective function along the update direction. This allows for more adaptive and potentially faster convergence.

D. Update Step

After computing the step-size, we perform:

$$x_{t+1} = x_t + \alpha(w_t - x_t).$$

This completes our algorithm.

Algorithm 1 Frank-Wolfe for Matrix Completion

Require: Start point $x_0 \in C$

- 1: **for** $k = 0, 1, \dots$ **do**
 - 2: Compute (u_k, v_k) , the top singular vectors of $-\nabla f(x_k)$
 - 3: $s_k \leftarrow \delta u_k v_k^T$
 - 4: $x_{k+1} \leftarrow x_k + \alpha_k(s_k - x_k)$
 - 5: **end for**
-

V. ILLUSTRATIVE EXPERIMENT

To evaluate the performance of the Frank-Wolfe algorithm, I conduct numerical experiments on two distinct datasets, both being slim versions of their original counterparts. These datasets have been carefully curated to maintain relevance while reducing sparsity, allowing for efficient and meaningful analysis. A brief overview of each dataset is provided below:

MovieLens Slim is a refined subset of the widely recognized MovieLens dataset, developed by the GroupLens Research team at the University of Minnesota. It contains 100,000 user ratings, ranging from 1 to 5, on a carefully selected subset of movies, ensuring a more compact yet representative sample. By filtering out less informative data, this version reduces sparsity while preserving the core structure essential for evaluating recommender systems. This dataset is particularly valuable for research in personalized recommendations, enabling robust experimentation while maintaining manageable data complexity.

X-Wines Slim is a structured subset of the comprehensive X-Wines dataset, designed to facilitate research in recommender systems and machine learning. This version includes 1,007 wines from 31 countries across six distinct wine types, accompanied by 150,000 user ratings from 10,561 anonymized users, with ratings on a scale from 1 to 5. To enhance data robustness, each user has rated at least five wines, and each wine has received a minimum of five ratings. Additionally, the dataset is structured to ensure that no user provides multiple ratings for the same wine, simplifying analysis and making it more suitable for recommendation model evaluation.

For both datasets, the algorithms were run for a maximum of 2000 iterations with two different step sizes. For each configuration, we recorded the loss values, R^2 scores for both the training and test sets, duality gap, CPU time, and the number of iterations. Figures 3 to 6 illustrate the loss values and R^2 scores as functions of the number of iterations.

All the results obtained in our experiments demonstrate convergence. Furthermore, as shown in Figure 7 and 8 in terms of CPU time per iteration, the diminishing step size proves to be more efficient than the line search method.

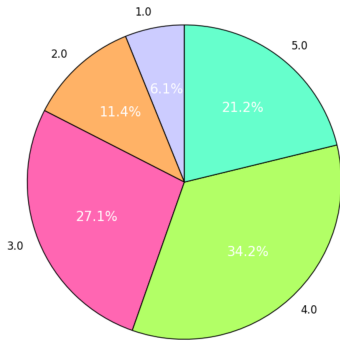


Fig. 1: Class Distribution in MovieLens Dataset

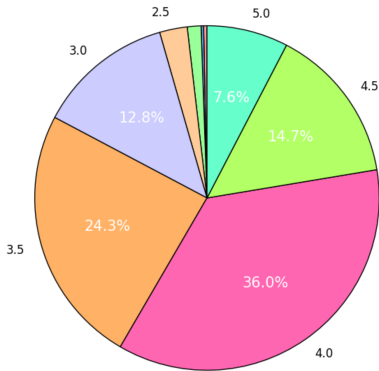


Fig. 2: Class Distribution in X-Wines Dataset

VI. RESULTS

For the MovieLens dataset, both step-size strategies resulted in similar accuracy levels, suggesting that step-size selection had little impact on this dataset. However, for the X-Wines dataset, the line search strategy achieved better accuracy than the diminishing step-size, indicating that an adaptive approach can be beneficial in certain cases. Despite its advantages in accuracy, the line search method was consistently more computationally expensive, requiring more CPU time per iteration than the diminishing step-size. This trade-off highlights that the choice of step-size strategy depends on the specific dataset and available computational resources.

As shown in Table 1, another key observation was that overall accuracy was lower than expected, which may be due to the non-uniform class distribution in the datasets, Figure 1 and Figure 2. Specifically, classes with fewer samples exhibited lower accuracy, likely because the model had less training data to learn meaningful patterns. This class imbalance suggests that dataset characteristics play a crucial role in algorithm performance, affecting the ability of the model to generalize across different categories.

These results highlight the importance of selecting the right step-size strategy based on dataset properties and computational constraints. While the Frank-Wolfe algorithm demonstrated effective optimization capabilities, accuracy was influenced by dataset structure and class distribution, which should be considered in future applications.

TABLE 1: Accuracy for MovieLens and X-Wines dataset

	MovieLens		X-Wines	
	Train	Test	Train	Test
Diminishing Step-Size	0.64	0.37	0.4	0.25
Exact Line Search	0.65	0.41	0.7	0.3

VII. CONCLUSION

In this study, we implemented the Frank-Wolfe algorithm for optimizing recommender systems using matrix completion. The algorithm leveraged an efficient linear minimization oracle (LMO), enabling iterative low-rank matrix approximation while maintaining computational feasibility. We evaluated two step-size strategies, reduced line and exact line search, in the MovieLens and X-Wines datasets. While line search achieved better accuracy on X-Wines, it was computationally more expensive than the diminishing step-size.

Our results indicate that accuracy was affected by the imbalance of data set class, with lower performance in underrepresented classes. To improve convergence and accuracy, future work could explore Frank-Wolfe variants and projection-based optimization methods. Despite some limitations, our findings highlight the potential of Frank-Wolfe in recommender systems while suggesting areas for further enhancement.

REFERENCES

- [1] Immanuel M. Bomze, Francesco Rinaldi, and Damiano Zeffiro. Frank-wolfe and friends: A journey into projection-free first-order optimization methods. *4OR*, 19(3):313–345, 2021.
- [2] E. J. Candès and B. Recht. Exact matrix completion via convex optimization. *Foundations of Computational Mathematics*, 9(6):717–772, 2009.
- [3] Marguerite Frank and Philip Wolfe. An algorithm for quadratic programming. *Naval Research Logistics Quarterly*, 3(1-2):95–110, 1956.
- [4] Robert M. Freund, Paul Grigas, and Rahul Mazumder. An extended frank-wolfe method with in-face directions, and its application to low-rank matrix completion. *SIAM Journal on Optimization*, 27(1):319–346, 2017.
- [5] Martin Jaggi. Revisiting frank-wolfe: Projection-free sparse convex optimization. In *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*, pages 427–435, 2013.
- [6] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.

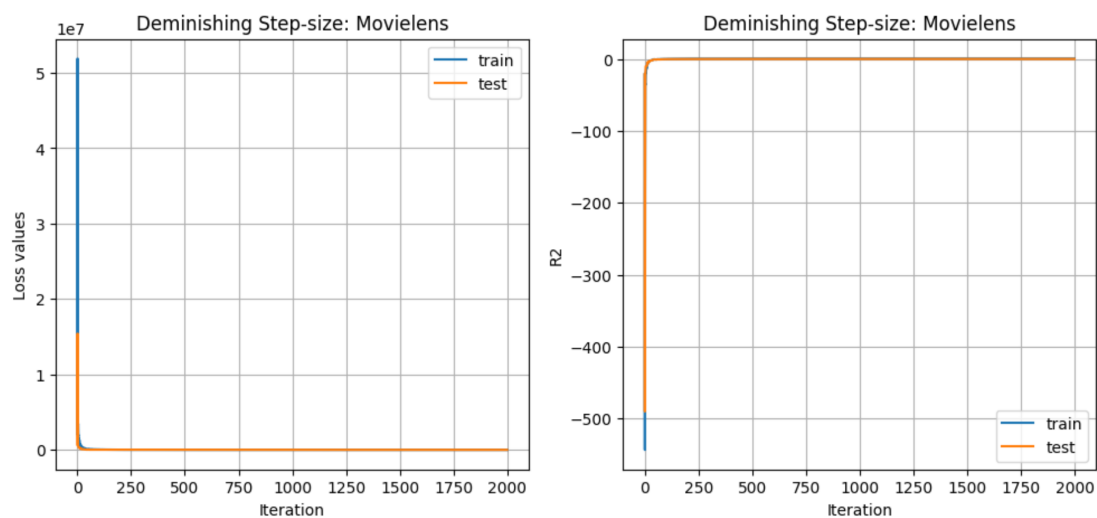


Fig. 3: Frank-Wolfe algorithm using Diminishing step-size: Movielens

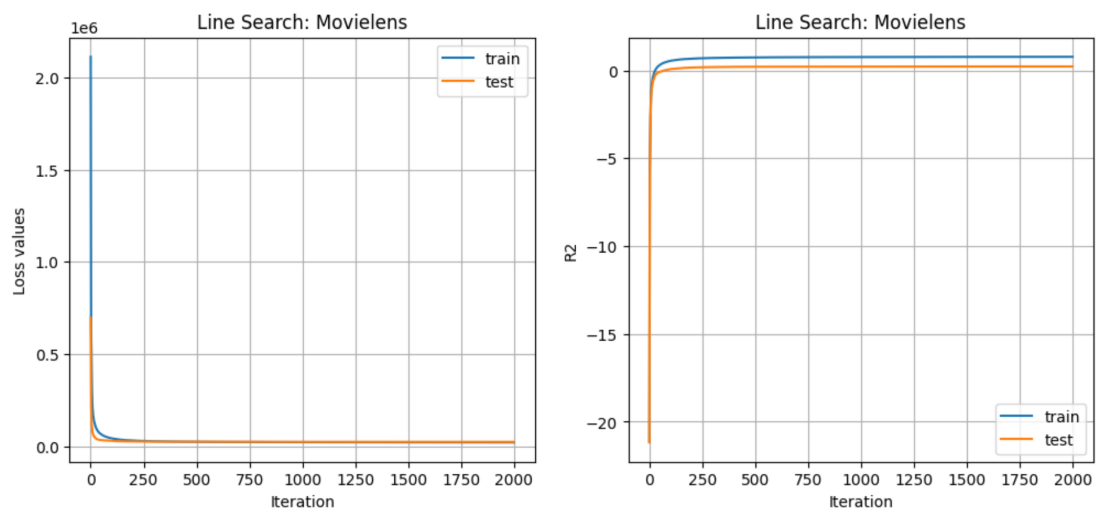


Fig. 4: Frank-Wolfe algorithm using exact line search: Movielens

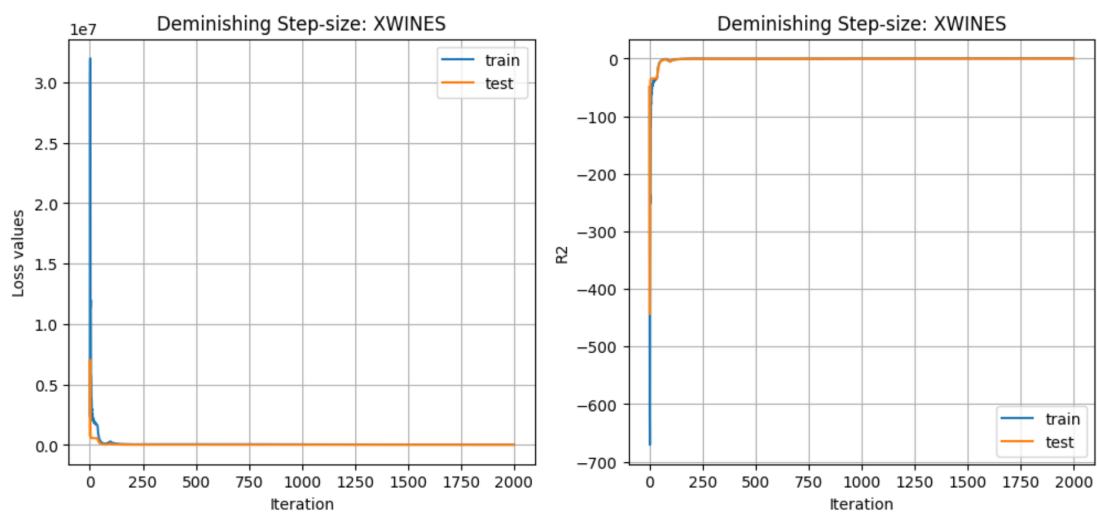


Fig. 5: Frank-Wolfe algorithm using Diminishing step-size: X-Wines

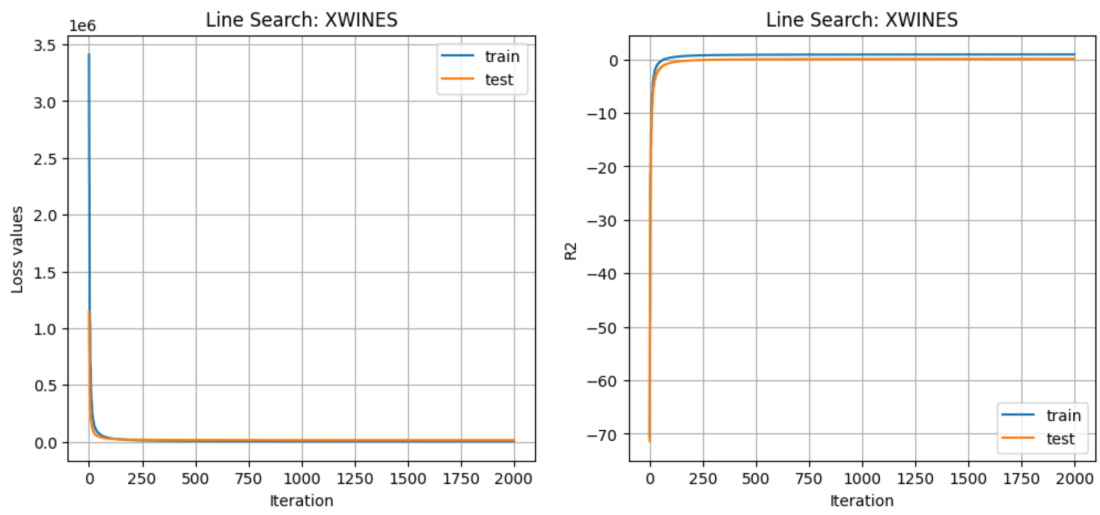


Fig. 6: Frank-Wolfe algorithm using Exact line search: X-Wines

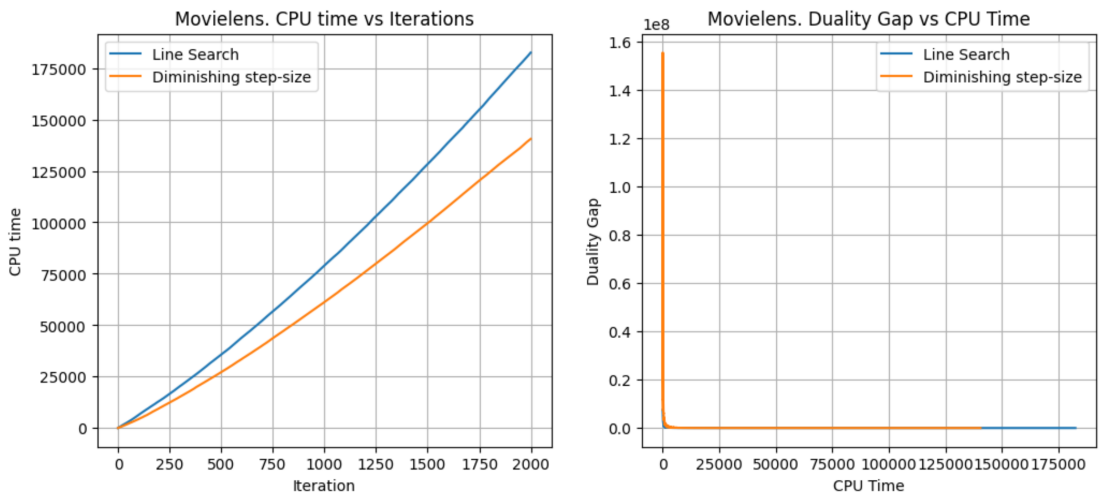


Fig. 7: Line Search vs Diminishing Step Size: Movielenz

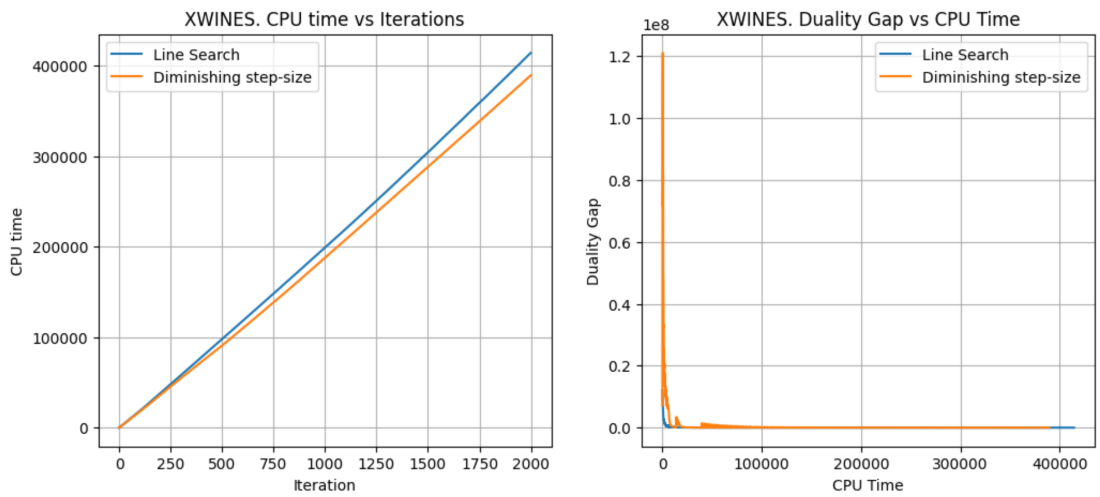
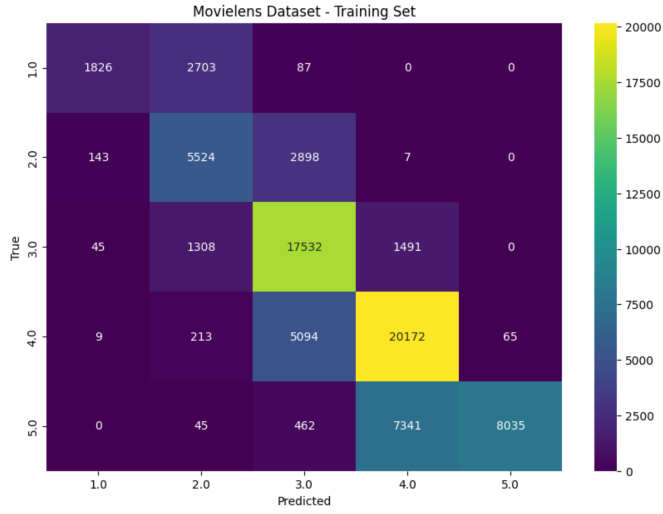
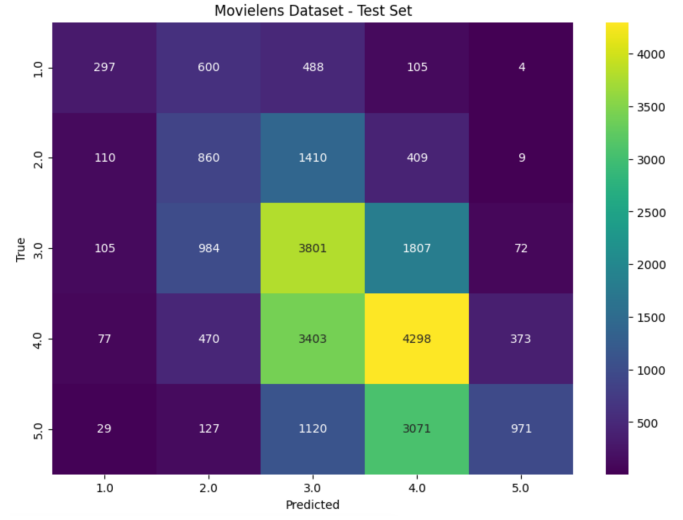


Fig. 8: Line Search vs Diminishing Step Size: X-Wines



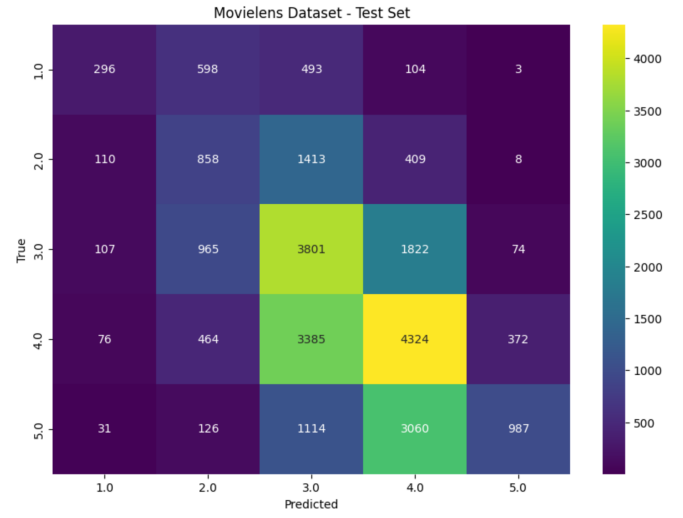
(a) Diminishing Step-Size: MovieLens Train



(b) Diminishing Step-Size: MovieLens Test



(c) Exact Line Search: MovieLens Train

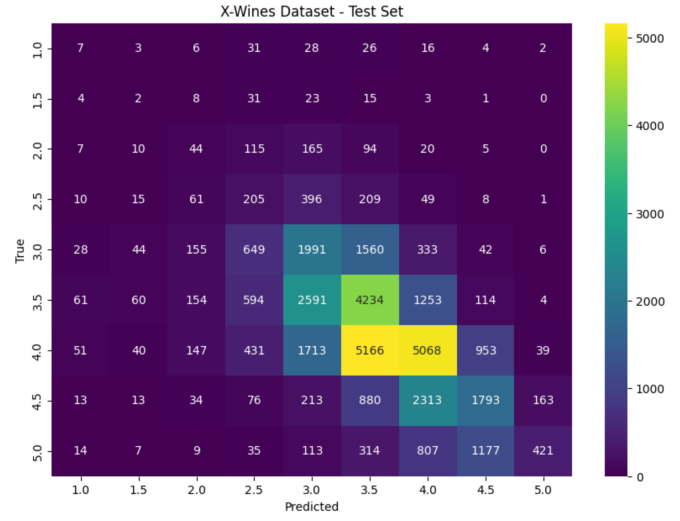


(d) Exact Line Search: MovieLens Test

Fig. 9: Confusion matrices for the MovieLens dataset using Diminishing Step-Size and Exact Line Search (Train/Test).



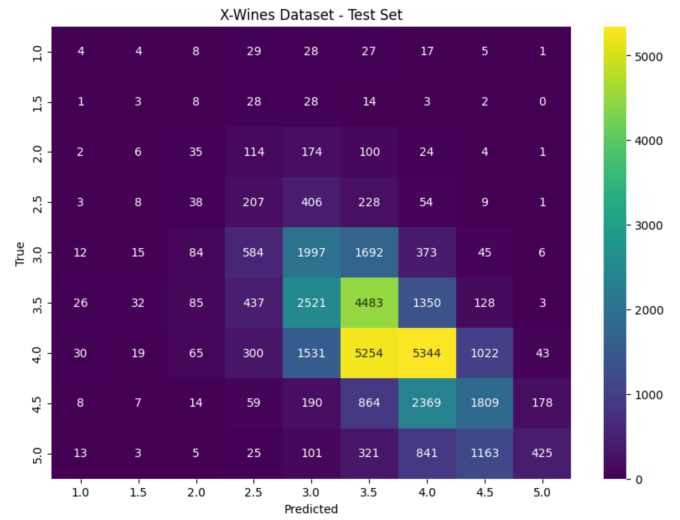
(a) Diminishing Step-Size: X-Wines Train



(b) Diminishing Step-Size: X-Wines Test



(c) Exact Line Search: X-Wines Train



(d) Exact Line Search: X-Wines Test

Fig. 10: Confusion matrices for the X-Wines dataset using Diminishing Step-Size and Exact Line Search (Train/Test).